

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – PSEUDOCODE WRITING TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	Assessment:	Pseudocode Writing task
Weightage:	30%	Total Marks:	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	ABDUL WAHID. H	Reg. No.:	192321079
Section / Batch:	B.TECH IT	Date:	24/08/2026

Instructions to Students

- Identify the relevant concepts of DMA-based data transfer, vectored interrupts, I/O mapping techniques, bus arbitration, and hybrid I/O communication.
- Analyze the requirements of different I/O devices by considering CPU involvement, interrupt priority, latency, bandwidth, throughput, and transfer speed.
- Apply appropriate I/O techniques such as DMA, interrupt-driven I/O, vectored interrupts, memory-mapped I/O, I/O-mapped I/O, and bus arbitration to develop efficient solutions.
- Develop pseudocode algorithms for DMA data transfer, prioritized interrupt handling, dynamic I/O mapping selection, bus arbitration, and hybrid polling–DMA communication.
- Evaluate the proposed I/O mechanisms by interpreting CPU utilization, data transfer efficiency, interrupt response time, communication performance, resource allocation, and overall system suitability.

QUESTIONS

1. A real-time embedded system continuously receives large blocks of data from a high-speed I/O device. Write pseudocode to design a DMA-based data transfer algorithm that transfers data directly from the I/O device to main memory with minimal CPU involvement. The algorithm should initialize the DMA controller, configure source and destination addresses, specify the transfer size, initiate the transfer, and handle completion and error interrupts.
2. An embedded system is connected to multiple I/O devices, each capable of generating interrupts with different priority levels. Develop pseudocode for an interrupt handling mechanism that identifies and prioritizes vectored interrupts, determines the interrupting device, and dispatches the appropriate interrupt service routine. The algorithm should ensure that high-priority interrupts are serviced efficiently while maintaining proper handling of lower-priority requests.
3. A computer system communicates with different I/O devices having varying latency, bandwidth, and datatransfer requirements. Write pseudocode for a dynamic I/O selection mechanism that chooses between memory-mapped I/O and I/O-mapped I/O based on the performance requirements of each device. The algorithm should evaluate latency and bandwidth requirements and select the mapping technique that provides efficient communication.
4. A computer system contains multiple devices that simultaneously request access to high-speed communication interfaces such as PCIe, USB, and Thunderbolt. Design pseudocode for a bus arbitration algorithm that manages competing access requests and allocates the appropriate interface to each device. The algorithm should consider device priority, bandwidth requirements, interface availability, and fairness to ensure efficient utilization of communication resources.
5. A real-time embedded system communicates with both low-speed devices, such as simple sensors, and high-speed devices, such as data acquisition units and storage devices. Write pseudocode for a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupt-driven communication for high-speed devices. The algorithm should identify the device type, select the appropriate I/O technique, initiate data transfer, and handle completion and error conditions while minimizing CPU involvement.

Code of Conduct

*I **ABDUL WAHID. H / 192321079** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: ABDUL WAHID. H

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Q. No	Problem Understanding (20)	Algorithm Design (30)	Use of Control Structures (20)	Pseudocode Structure (10)	Completeness (20)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/30	/ 20	/ 10	/ 20	/ 100

Course Coordinator

Faculty Incharge

1. DMA-Based Data Transfer Algorithm

Purpose: Transfer a large block of data directly from a high-speed I/O device to main memory with minimal CPU involvement. The CPU initializes the DMA controller, while DMA performs the actual data movement and interrupts the CPU only on completion or error.

```
ALGORITHM DMA_Transfer

BEGIN
    Initialize DMA_Controller
    Set Source_Address ← IO_Device_Data_Register
    Set Destination_Address ← Main_Memory_Address
    Set Transfer_Size ← Data_Block_Size
    Set Transfer_Direction ← IO_TO_MEMORY

    Enable DMA_Completion_Interrupt
    Enable DMA_Error_Interrupt

    Start DMA_Transfer

    WHILE DMA_Status = BUSY DO
        CPU performs other tasks
    END WHILE
END

INTERRUPT DMA_Completion_ISR

BEGIN
    Clear DMA_Completion_Flag
    Mark Transfer_Status ← SUCCESS
    Notify CPU / Application
END

INTERRUPT DMA_Error_ISR

BEGIN
    Read DMA_Error_Status
    Stop DMA_Transfer
    Clear Error_Flag
    Mark Transfer_Status ← ERROR
    Report Error
    Restart or Recover DMA if required
END
```

Explanation

- Source address identifies the I/O device data register, while destination address points to the main-memory buffer.
- The CPU does not copy the bulk data; the DMA controller performs the transfer.
- A completion interrupt notifies the CPU after the entire block is transferred.
- An error interrupt handles failed or invalid transfers.
- This minimizes CPU involvement and is suitable for continuous high-speed data.

2. Prioritized Vectored Interrupt Handling

Purpose: Identify the interrupting device, prioritize simultaneous requests, and dispatch the correct interrupt service routine (ISR). High-priority interrupts are serviced first, while lower-priority requests remain pending.

```
ALGORITHM Priority_Interrupt_Handler

BEGIN
    Initialize Interrupt_Controller
    Initialize Vector_Table
    Assign Priority to Each Device

    WHILE System_Running = TRUE DO

        IF Interrupt_Request = TRUE THEN

            Pending_List ← Read_Pending_Interrupts()

            Highest ← Select_Highest_Priority(Pending_List)

            Vector_Address ← Vector_Table[Highest]

            Save_CPU_Context

            Dispatch to Vector_Address

            Restore_CPU_Context

        END IF

    END WHILE
END

ISR Device_Handler(Device_ID)

BEGIN
    Acknowledge_Interrupt(Device_ID)
    Execute_Service_Routine(Device_ID)
    Clear_Interrupt(Device_ID)
    Return_From_Interrupt
END
```

Explanation

- The interrupt controller maintains pending interrupt requests and their priorities.
- The highest-priority pending device is selected first.
- Vectored interrupt handling directly obtains the ISR address from the vector table.
- Lower-priority requests remain pending and are serviced after higher-priority work is completed.
- This provides efficient interrupt response without repeatedly checking every device.

3. Dynamic I/O Mapping Selection

Purpose: Select memory-mapped I/O (MMIO) or I/O-mapped I/O (I/O space) according to device latency, bandwidth, and data-transfer requirements.

```
ALGORITHM Dynamic_IO_Mapping

INPUT:
    Device_Latency
    Device_Bandwidth
    Required_Latency
    Required_Bandwidth

BEGIN

    IF Device_Latency <= Required_Latency AND
       Device_Bandwidth >= Required_Bandwidth THEN

        Mapping ← MEMORY_MAPPED_IO

    ELSE IF Device_Bandwidth >= Required_Bandwidth THEN

        Mapping ← MEMORY_MAPPED_IO

    ELSE

        Mapping ← IO_MAPPED_IO

    END IF

    Configure_Device_Mapping(Mapping)

    RETURN Mapping

END
```

Improved Decision Rule

For a practical embedded/high-performance design, MMIO is preferred when the device requires fast, frequent register access or high-throughput communication. I/O-mapped I/O can be selected when the architecture provides a separate I/O address space and the device does not require the advantages of normal memory-style register access.

```
IF High_Speed = TRUE OR Low_Latency_Required = TRUE THEN
    Select MEMORY_MAPPED_IO
ELSE
    Select IO_MAPPED_IO
END IF
```

Explanation

- MMIO places device registers in the normal memory address space and is convenient for frequent load/store access.
- I/O-mapped I/O uses a separate I/O address space and dedicated I/O instructions where the processor supports them.
- The selection should be based on actual processor architecture and device requirements; the question does not provide numerical latency or bandwidth values.

4. Fair Bus Arbitration Algorithm

Purpose: Manage simultaneous requests from devices using PCIe, USB, and Thunderbolt while considering priority, bandwidth, interface availability, and fairness.

```
ALGORITHM Bus_Arbitration

BEGIN
    Initialize Request_Queue
    Initialize Interface_Status
    Initialize Device_Priorities

    WHILE System_Running = TRUE DO

        Collect all device requests

        FOR each Request in Request_Queue DO
            Calculate Score =
                Priority_Weight
                + Bandwidth_Requirement
                + Waiting_Time
        END FOR

        Select Request with highest Score

        IF Required_Interface_Available(Request) = TRUE THEN

            Allocate_Interface(Request)
            Allocate_Bandwidth(Request)

            Grant_Bus_Access(Request)

            Transfer_Data(Request)

            Release_Interface(Request)
            Remove Request from Queue

        ELSE
            Keep Request in Queue
            Try next eligible request
        END IF

        Increase Waiting_Time of all waiting requests

    END WHILE
END
```

Explanation

- Priority ensures important devices receive timely service.
- Bandwidth requirements help select an interface capable of supporting the transfer.
- Interface availability prevents two devices from using an unavailable resource simultaneously.
- Waiting time increases the score of waiting devices, preventing starvation.
- After a transfer, the interface is released so another device can use it.
- This provides a balance between priority-based service and fair allocation.

5. Hybrid Polling–DMA I/O Communication

Purpose: Use polling for low-speed devices such as simple sensors and DMA with interrupt-driven communication for high-speed devices such as data acquisition and storage devices, while minimizing CPU involvement.

```
ALGORITHM Hybrid_IO_Communication

BEGIN
    Initialize All_Devices

    FOR each Device DO

        IF Device.Type = LOW_SPEED THEN

            Device_Mode ← POLLING

            WHILE Device_Active = TRUE DO

                IF Device_Status = DATA_READY THEN
                    Data ← Read_Device()
                    Process(Data)
                    Clear_Device_Status()
                END IF

            END WHILE

        ELSE IF Device.Type = HIGH_SPEED THEN

            Device_Mode ← DMA

            Configure_DMA_Source(Device)
            Configure_DMA_Destination(Main_Memory_Buffer)
            Configure_DMA_Size(Data_Block_Size)

            Enable_DMA_Completion_Interrupt
            Enable_DMA_Error_Interrupt

            Start_DMA

        END IF

    END FOR

END

INTERRUPT DMA_Completion_ISR

BEGIN
    Clear_Completion_Flag
    Process_Completed_Buffer
    Start_Next_DMA_Transfer
END

INTERRUPT DMA_Error_ISR

BEGIN
    Read_Error_Status
    Stop_DMA
    Handle_Error
    Restart_DMA if required
END
```


Explanation

- Low-speed sensors are polled because their data rate is small and periodic status checking is simple.
- High-speed devices use DMA so large data blocks move directly to memory without continuous CPU copying.
- DMA completion interrupts notify the CPU only when a block is ready.
- DMA error interrupts handle transfer failures.
- This hybrid approach reduces CPU overhead while maintaining responsive communication with both device types.